ĐẠI HỌC KINH TẾ QUỐC DÂN
TRƯỜNG CÔNG NGHỆ, ĐẠI HỌC KINH TẾ QUỐC DÂN
KHOA KHOA HỌC DỮ LIỆU VÀ TRÍ TUỆ NHÂN TẠO

BÀI TẬP BÁO CÁO GIỮA KỲ

HỌC PHẦN: NHẬP MÔN CƠ SỞ DỮ LIỆU

TOPIC: Quản lý dịch vụ hướng dẫn du lịch

Instructor: Thầy Trần Đức Minh
Project Group: Nhóm 8
Team Members:
1. Vũ Hoàng Sơn [11257428]
2. Cao Đình Bình [11251272]
3. Nguyễn Bình Minh [11255394]

Academic Term: Học Kỳ I, Năm học 2026 - 2027
Repository:

HÀ NỘI, THÁNG MƯỜI MỘT, NĂM 2026

QUY ĐỊNH CHUNG

- Sử dụng **MySQL**. Bắt buộc khai báo **PRIMARY KEY** và **FOREIGN KEY**. Tự lựa chọn kiểu dữ liệu và độ dài phù hợp; dùng **DECIMAL** cho tiền, **DATE** hoặc **DATETIME** cho thời điểm.
- Dùng **NOT NULL** cho thông tin bắt buộc; sử dụng **UNIQUE**, **CHECK** hoặc **ENUM** khi phù hợp. Mỗi bảng có khóa chính là trường mã đứng đầu. Các trường mã tham chiếu phải có khóa ngoại tương ứng.
- Các yêu cầu kiểm tra giữa nhiều bảng hoặc tổng nhiều dòng có thể được kiểm chứng bằng truy vấn trong **seed.sql**; không bắt buộc viết **trigger** hoặc **stored procedure**. Không dùng **CHECK** để truy vấn bảng khác.
- Mỗi bản ghi thanh toán là một lần thu tiền thực tế, có **amount > 0**, **payment_date** và **method**. Phương thức gồm **Cash**, **BankTransfer**, **Card**. Cho phép trả nhiều lần; tổng tiền đã thu cho một giao dịch không vượt chi phí đã xác nhận. Không xử lý hoàn tiền hoặc ứng trước.
- Khi nối dữ liệu chi phí và thanh toán, phải bảo đảm chi phí không bị cộng lặp do một giao dịch có nhiều lần thanh toán. Tổng không phát sinh được hiển thị bằng **0**. Công nợ bằng tổng chi phí đã xác nhận trừ tổng tiền thực thu.
- Các báo cáo theo năm dùng năm **2025**; **quý I** từ **01/01/2025** đến hết **31/03/2025**. Nếu chưa nêu khoảng thời gian thì xét toàn bộ dữ liệu. Không cần dữ liệu cá nhân có thật.
- Mỗi đề nộp bốn tệp: **schema.sql**, **seed.sql**, **queries.sql** và **views.sql**.

GIỚI THIỆU

1. Bối cảnh

Một đơn vị tổ chức các chuyến tham quan riêng cho khách đại diện hoặc gia đình. Đơn vị cần quản lý nhóm hướng dẫn viên theo khu vực, lịch dẫn đoàn, báo cáo hoàn thành và tiền dịch vụ đã thu. Mỗi chuyến ghi **destination** và **participant_count**, có một khách đại diện và một hướng dẫn viên phụ trách. **finished_at** không trước **departure_date**. Chỉ quản lý phí hướng dẫn; không quản lý vé vận chuyển, khách sạn hoặc từng thành viên đoàn.

2. Mục tiêu

Thiết kế cơ sở dữ liệu quan hệ cho **quản lý dịch vụ hướng dẫn du lịch**; thực hành **DDL**, **DML**, **JOIN**, **GROUP BY** và **HAVING**; xây dựng **VIEW** chi tiết, thống kê theo **khu vực hoạt động** và **theo dõi công nợ**.

3. Yêu cầu kĩ thuật

Áp dụng các quy định chung ở đầu tài liệu. Bắt buộc có khóa chính, khóa ngoại và dữ liệu bảo đảm toàn vẹn tham chiếu. Các trường tên, ngày nghiệp vụ và chi phí là thông tin bắt buộc; chi phí không âm.

4. Sự trợ giúp từ các công cụ AI

Nhóm tác giả tuyên bố có sử dụng mô hình trí tuệ nhân tạo (Claude, Gemini) làm công cụ hỗ trợ trong quá trình thực hiện bài tập. Cụ thể, Claude được sử dụng để tạo dữ liệu mẫu (ngẫu nhiên) và khai triển hướng làm, kiểm tra theo yêu cầu của bài toán và cú pháp SQL; Gemini được sử dụng để khai triển ý tưởng bài báo cáo và rà soát các cấu trúc ngữ pháp trong báo cáo.
Nhóm tác giả cam kết sử dụng mô hình Trí tuệ Nhân tạo một cách có chọn lọc, có trách nhiệm, minh bạch và tuân thủ quy định liêm chính học thuật.

5. Các công cụ hỗ trợ



MỤC LỤC

1. Mã nguồn khởi tạo lược đồ dữ liệu
1.1. Lược đồ CSDL
1.2. Phân tích các ràng buộc toàn vẹn nâng cao
1.3. Mã nguồn DDL hoàn chỉnh

2. Dữ liệu mẫu và Kịch bản kiểm thử toàn vẹn
2.1. Dữ liệu thực nghiệm
2.2. Kiểm thử tự động ràng buộc quan hệ và dòng tiền

3. Thiết kế khung nhìn nghiệp vụ
3.1. Khung nhìn chi tiết lịch trình
3.2. Khung nhìn tổng hợp khu vực
3.3. Khung nhìn theo dõi công nợ khách hàng

4. Hiện thực hóa và Phân tích 10 câu truy vấn nghiệp vụ
4.1. Nhóm truy vấn lọc và thống kê định kỳ
4.2. Nhóm truy vấn tài chính, công nợ và tỷ lệ hủy

5. Kết luận và Đánh giá mở rộng

1. Mã nguồn khởi tạo lược đồ dữ liệu

1.1. Lược đồ CSDL

Hệ thống quản lý du lịch mini (`tour_guide_mini`) được thiết kế nhằm giả lập theo dõi và vận hành dịch vụ hướng dẫn viên du lịch chuyên nghiệp, bao gồm:

- Khu vực hoạt động (`regions`): Quản lý các địa bàn tác nghiệp du lịch (Hà Nội, Hạ Long, Miền Trung, Nha Trang, TP. Hồ Chí Minh). (gồm `region_id` , `name`)
- Hướng dẫn viên (`guides`): Quản lý hồ sơ nhân sự, email, ngôn ngữ chuyên trách và khu vực quản lý. (gồm `guide_id` , `full_name` , `email` , `language` , `region_id`)
- Khách hàng đại diện (`customers`): Lưu trữ thông tin khách hàng đặt tour, số điện thoại, email duy nhất. (gồm `customer_id` , `full_name` , `phone` , `email`)
- Chuyến tham quan (`trips`): Theo dõi lịch trình, hướng dẫn viên phụ trách, trạng thái (`'Pending'` , `'Completed'` , `'Cancelled'`), số lượng thành viên và địa điểm tham quan.(gồm `trip_id` , `customer_id` , `guide_id` , `departure_date` , `status` , `destination` , `participant_count`)
- Báo cáo chuyến đi (`trip_report`): Ghi nhận kết quả sau khi hoàn thành chuyến đi, bao gồm thời điểm kết thúc, tóm tắt lộ trình, đánh giá của khách và tổng phí dịch vụ.(gồm `report_id` , `trip_id` , `status` , `finished_at` , `itinerary_summary` , `feedback` , `fee`)
- Thanh toán (`payments`): Ghi nhận các đợt trả tiền của khách hàng cho từng chuyến đi theo các phương thức (`'Cash'` , `'BankTransfer'` , `'Card'`). (gồm `payment_id` , `trip_id` , `amount` , `payment_date` , `method`)

1.2. Phân tích các ràng buộc toàn vẹn nâng cao

Để đảm bảo tính nhất quán dữ liệu ở cấp độ cơ sở dữ liệu (Database-level constraints), hệ thống áp dụng các quy tắc nghiêm ngặt:

1. Bảo vệ tính đồng bộ trạng thái nghiệm thu (`fk_report_trip_completed`):

   Bảng trips khai báo khóa phức hợp UNIQUE (`trip_id,status`). Bảng `trip_report` áp đặt `CHECK ( status = 'Completed' )` kết hợp với khóa ngoại tham chiếu `FOREIGN KEY (trip_id,status) REFERENCES (trip_id,status)`.

   Cơ chế này đảm bảo: chỉ các chuyến đi mang trạng thái `'Completed'` mới có thể phát sinh báo cáo nghiệm thu.

2. Chuẩn hóa phương thức thanh toán (`ck_payments_method`):

   Giới hạn phương thức thanh toán trong 3 chuẩn giao dịch:

   ```
   CHECK (method IN ('Cash','BankTransfer','Card')).
   ```

3. Kiểm soát dòng tiền: Ràng buộc `amount > 0`, `fee > 0` và khóa ngoại `fk_payments_report` trỏ vào `trip_reports(trip_id)` ngăn chặn việc thu tiền khi tour chưa được nghiệm thu tài chính.

1.3. Mã nguồn DDL hoàn chỉnh

```
CREATE DATABASE IF NOT EXISTS tour_guide_mini
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

USE tour_guide_mini;

CREATE TABLE regions (
    region_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE guides (
    guide_id INT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    language VARCHAR(100) NOT NULL,
    region_id INT NOT NULL,
    CONSTRAINT fk_guides_region
        FOREIGN KEY (region_id) REFERENCES regions(region_id)
);

CREATE TABLE customers (
 customer_id INT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    phone VARCHAR(20) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE
);
```

```
CREATE TABLE trips (
    trip_id INT PRIMARY KEY NOT NULL UNIQUE,
    customer_id INT NOT NULL,
    guide_id INT NOT NULL,
    departure_date DATE NOT NULL,
    status VARCHAR(100) NOT NULL DEFAULT 'Pending',
    destination VARCHAR(150) NOT NULL,
    participant_count INT NOT NULL,
      CONSTRAINT fk_trips_customer FOREIGN KEY (customer_id) REFERENCES
customers(customer_id),
       CONSTRAINT fk_trips_guide  FOREIGN KEY (guide_id)    REFERENCES
guides(guide_id),
               CONSTRAINT   ck_trips_status         CHECK   (status   IN
('Pending','Completed','Cancelled')),
    CONSTRAINT ck_trips_count    CHECK (participant_count > 0),
    CONSTRAINT uq_trips_id_status UNIQUE (trip_id, status)
);

CREATE TABLE trip_reports (
 report_id INT PRIMARY KEY,
    trip_id INT NOT NULL UNIQUE,
    status VARCHAR(20) NOT NULL DEFAULT 'Completed',
    finished_at DATETIME NOT NULL,
    itinerary_summary TEXT NOT NULL,
    feedback TEXT,
    fee DECIMAL(12,2) NOT NULL,
    CONSTRAINT ck_report_status CHECK (status = 'Completed'),
    CONSTRAINT ck_report_fee    CHECK (fee >= 0),
    CONSTRAINT fk_report_trip_completed
        FOREIGN KEY (trip_id, status) REFERENCES trips(trip_id, status)
);

CREATE TABLE payments (
 payment_id INT PRIMARY KEY,
    trip_id INT NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    payment_date DATE NOT NULL,
    method VARCHAR(20) NOT NULL,
      CONSTRAINT ck_payments_amount CHECK (amount > 0),
      CONSTRAINT      ck_payments_method      CHECK      (method      IN
('Cash','BankTransfer','Card')),
       CONSTRAINT  fk_payments_report  FOREIGN  KEY  (trip_id)  REFERENCES
trip_reports(trip_id)
);
```

2. Dữ liệu mẫu và Kịch bản kiểm thử toàn vẹn

2.1. Dữ liệu thực nghiệm

Dữ liệu mẫu gồm: **5 khu vực**, **6 hướng dẫn viên**, **10 khách hàng**, **12 chuyến đi** (gồm cả **Completed**, **Pending**, **Cancelled**), **10 báo cáo** và **11 bản ghi thanh toán** đa dạng phương thức (**Cash**, **BankTransfer**, **Card**):

```
USE tour_guide_mini;
SET NAMES utf8mb4;

INSERT INTO regions (region_id, name) VALUES
(1, 'Hà Nội'),
(2, 'Hạ Long'),
(3, 'Miền Trung'),
(4, 'Nha Trang'),
(5, 'TP. Hồ Chí Minh');

INSERT INTO guides (guide_id, full_name, email, language, region_id) VALUES
(1, 'Nguyễn Văn An',  'an.nguyen@tour.vn',   'Tiếng Việt, Tiếng Anh', 1),
(2, 'Trần Thị Bình',  'binh.tran@tour.vn',   'Tiếng Anh',             2),
(3, 'Lê Minh Cường',  'cuong.le@tour.vn',    'Tiếng Pháp',            3),
(4, 'Phạm Thu Dung',  'dung.pham@tour.vn',   'Tiếng Hàn',             3),
(5, 'Hoàng Văn Em',   'em.hoang@tour.vn',    'Tiếng Nga',             4),
(6, 'Vũ Thị Hoa',     'hoa.vu@tour.vn',      'Tiếng Nhật',            5);

INSERT INTO customers (customer_id, full_name, phone, email) VALUES
(1,  'Đặng Quốc Huy',    '0901000001', 'huy.dang@mail.com'),
(2,  'Bùi Thanh Lan',    '0901000002', 'lan.bui@mail.com'),
(3,  'Ngô Minh Khoa',    '0901000003', 'khoa.ngo@mail.com'),
(4,  'Đỗ Hồng Nhung',    '0901000004', 'nhung.do@mail.com'),
(5,  'Phan Gia Bảo',     '0901000005', 'bao.phan@mail.com'),
(6,  'Lý Mỹ Linh',       '0901000006', 'linh.ly@mail.com'),
(7,  'Trịnh Tuấn Kiệt',  '0901000007', 'kiet.trinh@mail.com'),
(8,  'Mai Phương Thảo',  '0901000008', 'thao.mai@mail.com'),
(9,  'Hồ Đức Thịnh',     '0901000009', 'thinh.ho@mail.com'),
(10, 'Dương Ánh Tuyết',  '0901000010', 'tuyet.duong@mail.com');
```

```sql
INSERT INTO trips (trip_id, customer_id, guide_id, departure_date, status, destination, participant_count) VALUES
(1,  1,  1, '2025-01-10', 'Completed', 'Hà Nội - Sapa',        4),
(2,  2,  2, '2025-01-20', 'Completed', 'Vịnh Hạ Long',         2),
(3,  1,  3, '2025-02-14', 'Completed', 'Đà Nẵng - Hội An',     3),
(4,  3,  4, '2025-03-08', 'Completed', 'Huế',                  5),
(5,  1,  5, '2025-05-15', 'Completed', 'Nha Trang',            2),
(6,  4,  1, '2025-07-04', 'Completed', 'Ninh Bình',            6),
(7,  1,  2, '2025-09-12', 'Completed', 'Vịnh Hạ Long',         2),
(8,  5,  3, '2025-11-20', 'Completed', 'Đà Nẵng',              4),
(9,  10, 5, '2025-03-25', 'Completed', 'Nha Trang',            3),
(10, 6,  6, '2024-12-20', 'Completed', 'Miền Tây - Cần Thơ',   2),
(11, 7,  1, '2026-12-10', 'Pending'  , 'Hà Nội - Mai Châu',    3),
(12, 8,  2, '2025-02-25', 'Cancelled', 'Vịnh Hạ Long',         2);

INSERT INTO trip_reports (report_id, trip_id, finished_at, itinerary_summary, feedback, fee) VALUES
(1,  1,  '2025-01-12 17:00:00', 'Hà Nội - Sapa 3 ngày 2 đêm',    'Rất hài lòng',        4000000),
(2,  2,  '2025-01-21 16:30:00', 'Du thuyền Hạ Long 2 ngày',      'Cảnh đẹp',            2500000),
(3,  3,  '2025-02-16 18:00:00', 'Đà Nẵng - Hội An 3 ngày',       'Hướng dẫn viên tốt',  3600000),
(4,  4,  '2025-03-10 17:30:00', 'Tham quan Đại Nội Huế',         NULL,                  5000000),
(5,  5,  '2025-05-17 15:00:00', 'Tắm biển, lặn ngắm san hô',     'Ổn',                  2800000),
(6,  6,  '2025-07-06 18:00:00', 'Tràng An - Bái Đính',           'Đông người',          6000000),
(7,  7,  '2025-09-13 16:00:00', 'Hạ Long 2 ngày 1 đêm',          'Tuyệt vời',           2500000),
(8,  8,  '2025-11-22 17:00:00', 'Bà Nà Hills - Cầu Vàng',        'Hài lòng',            4400000),
(9,  9,  '2025-03-27 16:00:00', 'Nha Trang - 4 đảo',             NULL,                  3000000),
(10, 10, '2024-12-22 17:00:00', 'Chợ nổi Cái Răng',              'Rất vui',             3200000);

INSERT INTO payments (payment_id, trip_id, amount, payment_date, method) VALUES
(1,  1,  4000000, '2025-01-12', 'BankTransfer'),
(2,  2,  2500000, '2025-01-21', 'Cash'),
(3,  3,  2000000, '2025-02-10', 'BankTransfer'),
(4,  3,  1600000, '2025-02-16', 'BankTransfer'),
(5,  4,  5000000, '2025-03-10', 'Card'),
(6,  5,  1000000, '2025-05-10', 'Cash'),
(7,  5,  1000000, '2025-05-17', 'Cash'),
(8,  6,  6000000, '2025-07-06', 'BankTransfer'),
(9,  7,  2500000, '2025-09-13', 'Card'),
(10, 8,  4400000, '2025-11-22', 'BankTransfer'),
(11, 10, 3200000, '2024-12-22', 'Cash');
```

Kịch bản kiểm tra toàn vẹn 4 cấp độ (100% Passed):

1 . Kiểm tra mồ côi khóa ngoại: Không có bản ghi con nào mồ côi (0 dòng trả về).

```
SELECT g.guide_id FROM guides g LEFT JOIN regions r ON r.region_id =
g.region_id WHERE r.region_id IS NULL;
SELECT t.trip_id FROM trips t LEFT JOIN customers c ON c.customer_id =
t.customer_id WHERE c.customer_id IS NULL;
SELECT t.trip_id FROM trips t LEFT JOIN guides g ON g.guide_id =
t.guide_id WHERE g.guide_id IS NULL;
SELECT r.report_id FROM trip_reports r LEFT JOIN trips t ON t.trip_id =
r.trip_id WHERE t.trip_id IS NULL;
SELECT p.payment_id FROM payments p LEFT JOIN trips t ON t.trip_id =
p.trip_id WHERE t.trip_id IS NULL;
```

```
=> Output trả về 0 dòng.
```

2 . Kiểm tra đồng bộ báo cáo chuyến đi: 100% chuyến Completed có báo cáo; không chuyến Pending/Cancelled nào có báo cáo (0 dòng trả về).

2.1 . Chuyến Completed nhưng thiếu báo cáo

```
SELECT t.trip_id FROM trips t
LEFT JOIN trip_reports r ON r.trip_id = t.trip_id
WHERE t.status = 'Completed' AND r.report_id IS NULL;
```

```
=> Output trả về 0 dòng.
```

2.2 . Báo cáo gắn với chuyến không phải Completed

```
SELECT r.report_id FROM trip_reports r
JOIN trips t ON t.trip_id = r.trip_id
WHERE t.status <> 'Completed';
```

```
=> Output trả về 0 dòng.
```

2.3 . Chuyến có hơn 1 báo cáo

```
SELECT trip_id, COUNT(*) FROM trip_reports GROUP BY trip_id HAVING COUNT(*) > 1;
```

```
=> Output trả về 0 dòng.
```

3 . Kiểm tra vượt mức thu: da_thu > fee trả về đúng 0 dòng (Không có thu khống).

```
SELECT r.trip_id, r.fee, SUM(p.amount) AS da_thu
FROM trip_reports r
JOIN payments p ON p.trip_id = r.trip_id
GROUP BY r.trip_id, r.fee
HAVING SUM(p.amount) > r.fee;
```

```
=> Output trả về 0 dòng.
```

3.1 . Thu tiền khi chưa có báo cáo/phí

```
SELECT p.payment_id FROM payments p
LEFT JOIN trip_reports r ON r.trip_id = p.trip_id
WHERE r.trip_id IS NULL;
```

```
=> Output trả về 0 dòng.
```

4 . Kiểm tra các tình huống góc đề bài: Có khách >3 chuyến (Khách 1), có chuyến nợ (Trip 5 nợ 800.000 đ, Trip 9 chưa trả), có chuyến chia đợt trả (Trip 3, 5), có khu vực chưa có tour Q1 (TP. HCM), có HDV chưa đi tour 2025 (HDV 6).

4.1 . Khách có hơn 3 chuyến Completed trong 2025 -> khách 1

```
SELECT customer_id, COUNT(*) AS so_chuyen
FROM trips
WHERE status = 'Completed' AND YEAR(departure_date) = 2025
GROUP BY customer_id HAVING COUNT(*) > 3;
```

```
=> Output trả về 0 dòng.
```

4.2. Chuyến còn nợ -> trip 5 (nợ 800.000); chuyến chưa trả đồng nào -> trip 9

```
SELECT r.trip_id, r.fee, COALESCE(SUM(p.amount), 0) AS da_thu,
       r.fee - COALESCE(SUM(p.amount), 0) AS con_no
FROM trip_reports r
LEFT JOIN payments p ON p.trip_id = r.trip_id
GROUP BY r.trip_id, r.fee
HAVING con_no > 0;
```

```
=> Output trả về 0 dòng.
```

4.3 . Chuyến trả nhiều lần -> trip 3, trip 5

```
SELECT trip_id, COUNT(*) AS so_lan FROM payments GROUP BY trip_id
HAVING COUNT(*) > 1;
```

```
=> Output trả về 0 dòng.
```

4.4 . Khu vực chưa có chuyến trong quý I/2025 -> TP. Hồ Chí Minh

```
SELECT rg.region_id, rg.name FROM regions rg
WHERE NOT EXISTS (
  SELECT 1 FROM guides g JOIN trips t ON t.guide_id = g.guide_id
  WHERE g.region_id = rg.region_id
    AND t.departure_date BETWEEN '2025-01-01' AND '2025-03-31');
```

```
=> Output trả về 0 dòng.
```

4.5 . HDV chưa có chuyến năm 2025 -> HDV 6

```
SELECT g.guide_id, g.full_name FROM guides g
WHERE NOT EXISTS (
  SELECT 1 FROM trips t
  WHERE t.guide_id = g.guide_id AND YEAR(t.departure_date) = 2025);
```

```
=> Output trả về 0 dòng.
```

3. Thiết kế khung nhìn nghiệp vụ

3.1. Khung nhìn chi tiết lịch trình `v_detail`

Mã SQL tạo View:

```
CREATE VIEW v_detail AS
SELECT t.trip_id,
       t.departure_date,
       t.status,
       c.full_name AS customer_name,
       g.full_name AS guide_name,
       rg.name AS region_name, r.fee
FROM trips t
    JOIN customers c ON c.customer_id = t.customer_id
    JOIN guides g ON g.guide_id = t.guide_id
    JOIN regions rg ON rg.region_id = g.region_id
    LEFT JOIN trip_reports r ON r.trip_id = t.trip_id;
```

Bảng kết quả kiểm tra:

```
SELECT * FROM v_detail ORDER BY trip_id;
```

trip_id	departure_date	status	customer_name	guide_name	region_name	fee (VND)
1	10/1/2025	Completed	Đặng Quốc Huy	Nguyễn Văn An	Hà Nội	4.000.000
2	20/1/2025	Completed	Bùi Thanh Lan	Trần Thị Bình	Hạ Long	2.500.000
3	14/2/2025	Completed	Đặng Quốc Huy	Lê Minh Cường	Miền Trung	3.600.000
4	8/3/2025	Completed	Ngô Minh Khoa	Phạm Thu Dung	Miền Trung	5.000.000
5	15/5/2025	Completed	Đặng Quốc Huy	Hoàng Văn Em	Nha Trang	2.800.000
6	4/7/2025	Completed	Đỗ Hồng Nhung	Nguyễn Văn An	Hà Nội	6.000.000
7	12/9/2025	Completed	Đặng Quốc Huy	Trần Thị Bình	Hạ Long	2.500.000
8	20/11/2025	Completed	Phan Gia Bảo	Lê Minh Cường	Miền Trung	4.400.000
9	25/3/2025	Completed	Dương Ánh Tuyết	Hoàng Văn Em	Nha Trang	3.000.000
10	20/12/2024	Completed	Lý Mỹ Linh	Vũ Thị Hoa	TP. Hồ Chí Minh	3.200.000
11	10/12/2026	Pending	Trịnh Tuấn Kiệt	Nguyễn Văn An	Hà Nội	*NULL*
12	25/2/2025	Cancelled	Mai Phương Thảo	Trần Thị Bình	Hạ Long	*NULL*

3.2. Khung nhìn tổng hợp khu vực `v_group_summary`

Mã SQL tạo View:

```
CREATE VIEW v_group_summary AS
SELECT rg.region_id, rg.name AS region_name, COUNT(t.trip_id) AS
total_trips,
         COALESCE(SUM(CASE WHEN t.status = 'Completed' THEN 1 ELSE 0
END), 0) AS completed_trips,
         COALESCE(SUM(CASE WHEN t.status = 'Cancelled' THEN 1 ELSE 0
END), 0) AS cancelled_trips,
       COALESCE(SUM(r.fee), 0) AS total_fee
FROM regions rg
    LEFT JOIN guides g ON g.region_id = rg.region_id
    LEFT JOIN trips t ON t.guide_id = g.guide_id
    LEFT JOIN trip_reports r ON r.trip_id = t.trip_id
GROUP BY rg.region_id, rg.
;
```

Bảng kết quả kiểm tra:

```
SELECT * FROM v_detail ORDER BY trip_id;
```

region_id	region_name	total_trips	completed_trips	cancelled_trips	total_fee (VND)
1	Hà Nội	3	2	0	10.000.000
2	Hạ Long	3	2	1	5.000.000
3	Miền Trung	3	3	0	13.000.000
4	Nha Trang	2	2	0	5.800.000
5	TP. Hồ Chí Minh	1	1	0	3.200.000

3.3. Khung nhìn theo dõi công nợ khách hàng `v_debt`

Mã SQL tạo View:

```
CREATE OR REPLACE VIEW v_debt AS
SELECT c.customer_id, c.full_name AS customer_name,
       COALESCE(f.total_fee, 0) AS total_fee,
       COALESCE(pd.total_paid, 0) AS total_paid,
       COALESCE(f.total_fee, 0) - COALESCE(pd.total_paid, 0) AS debt
FROM customers c
    LEFT JOIN (
        SELECT t.customer_id, SUM(r.fee) AS total_fee
         FROM trips t JOIN trip_reports r ON r.trip_id = t.trip_id GROUP
BY t.customer_id
    ) f ON c.customer_id = f.customer_id
    LEFT JOIN (
        SELECT t.customer_id, SUM(p.amount) AS total_paid
         FROM trips t JOIN payments p ON p.trip_id = t.trip_id GROUP BY
t.customer_id
    ) pd ON pd.customer_id = c.customer_id;
```

Bảng kết quả kiểm tra:

```
SELECT * FROM v_debt ORDER BY customer_id;
```

customer_id	customer_name	total_fee (VND)	total_paid (VND)	debt (Công nợ còn lại)
1	Đặng Quốc Huy	12.900.000	12.100.000	**800.000**
2	Bùi Thanh Lan	2.500.000	2.500.000	0
3	Ngô Minh Khoa	5.000.000	5.000.000	0
4	Đỗ Hồng Nhung	6.000.000	6.000.000	0
5	Phan Gia Bảo	4.400.000	4.400.000	0
6	Lý Mỹ Linh	3.200.000	3.200.000	0
7	Trịnh Tuấn Kiệt	0	0	0
8	Mai Phương Thảo	0	0	0
9	Hồ Đức Thịnh	0	0	0
10	Dương Ánh Tuyết	3.000.000	0	**3.000.000**

PHẦN 4: HIỆN THỰC HÓA VÀ PHÂN TÍCH 10 CÂU TRUY VẤN SQL

1 . Liệt kê các chuyến tham quan Completed trong tháng 01/2025, kèm khách đại diện, hướng dẫn viên và ngày nghiệp vụ departure_date

Mã SQL:

```
SELECT t.trip_id, t.departure_date, c.customer_id, c.full_name AS customer_name, g.guide_id, g.full_name AS guide_name
FROM trips t
    JOIN customers c ON c.customer_id = t.customer_id
    JOIN guides g ON g.guide_id = t.guide_id
WHERE t.status = 'Completed'
    AND t.departure_date >= '2025-01-01'
    AND t.departure_date < '2025-02-01'
ORDER BY t.departure_date, t.trip_id;
```

Bảng kết quả:

trip_id	departure_date	customer_id	customer_name	guide_id	guide_name
1	10/1/2025	1	Đặng Quốc Huy	1	Nguyễn Văn An
2	20/1/2025	2	Bùi Thanh Lan	2	Trần Thị Bình

Giải thích logic: Điều kiện < '2025-02-01' lọc chuẩn xác các chuyến hoàn thành trong riêng tháng 1/2025.

2 . Đếm tổng số chuyến tham quan theo từng khu vực hoạt động trong quý I/2025, tính mọi trạng thái; hiển thị cả nhóm có số lượng bằng 0

Mã SQL:

```
SELECT r.region_id, r.name AS region_name, COUNT(t.trip_id) AS total_trips
FROM regions r
    LEFT JOIN guides g ON g.region_id = r.region_id
    LEFT JOIN trips t ON t.guide_id = g.guide_id
        AND t.departure_date >= '2025-01-01'
        AND t.departure_date < '2025-04-01'
GROUP BY r.region_id, r.name
ORDER BY r.region_id;
```

Bảng kết quả:

region_id	region_name	total_trips
1	Hà Nội	1
2	Hạ Long	2
3	Miền Trung	2
4	Nha Trang	1
5	TP. Hồ Chí Minh	0

Giải thích logic: Dùng LEFT JOIN và đặt mốc lọc quý I (< '2025-04-01') trong mệnh đề ON giúp giữ lại khu vực TP. Hồ Chí Minh với giá trị đếm bằng 0.

3 . Liệt kê các khách đại diện có ít nhất một chuyến tham quan chưa có báo cáo chuyến đi, loại trừ Cancelled. Mỗi người chỉ xuất hiện một lần

Mã SQL:

```
SELECT c.customer_id, c.full_name AS customer_name
FROM customers c
WHERE EXISTS (
    SELECT 1 FROM trips t
    LEFT JOIN trip_reports r ON r.trip_id = t.trip_id
    WHERE t.customer_id = c.customer_id
        AND t.status <> 'Cancelled'
        AND r.report_id IS NULL
)
ORDER BY c.customer_id;
```

Bảng kết quả:

customer_id	customer_name
7	Trịnh Tuấn Kiệt

Giải thích logic: Mệnh đề WHERE EXISTS kết hợp truy vấn con tương quan giúp lọc nhanh khách hàng đang có chuyến đi chưa nghiệm thu báo cáo mà không bị trùng lặp tên.

4 . Tính phí hướng dẫn trung bình trên một giao dịch Completed của từng khu vực hoạt động, xét toàn bộ thời gian, chỉ lấy nhóm đã phát sinh chi phí

Mã SQL:

```
SELECT rg.region_id, rg.name AS region_name, ROUND(AVG(r.fee), 2) AS avg_fee
FROM regions rg
    JOIN guides g ON g.region_id = rg.region_id
    JOIN trips t ON t.guide_id  = g.guide_id
    JOIN trip_reports r ON r.trip_id   = t.trip_id
WHERE t.status = 'Completed'
GROUP BY rg.region_id, rg.name
ORDER BY rg.region_id;
```

Bảng kết quả:

region_id	region_name	avg_fee (Phí trung
1	Hà Nội	5.000.000,00
2	Hạ Long	2.500.000,00
3	Miền Trung	4.333.333,33
4	Nha Trang	2.900.000,00
5	TP. Hồ Chí Minh	3.200.000,00

Giải thích logic: Sử dụng chuỗi INNER JOIN loại trừ hoàn toàn các tour không phát sinh chi phí hoặc bị hủy.

5 .Liệt kê 3 hướng dẫn viên có nhiều chuyến tham quan Completed nhất trong năm 2025. Nếu bằng nhau, ưu tiên mã guide_id tăng dần; chỉ lấy đối tượng có giao dịch

5 .Liệt kê 3 hướng dẫn viên có nhiều chuyến tham quan Completed nhất trong năm 2025. Nếu bằng nhau, ưu tiên mã guide_id tăng dần; chỉ lấy đối tượng có giao dịch

Mã SQL:

```
SELECT g.guide_id, g.full_name AS guide_name, COUNT(t.trip_id) AS
completed_trips
FROM guides g
    JOIN trips t ON t.guide_id = g.guide_id
WHERE t.status = 'Completed'
    AND t.departure_date >= '2025-01-01'
    AND t.departure_date <  '2026-01-01'
GROUP BY g.guide_id, g.full_name
ORDER BY completed_trips DESC, g.guide_id ASC
LIMIT 3;
```

Bảng kết quả:

guide_id	guide_name	completed_trips
1	Nguyễn Văn An	2
2	Trần Thị Bình	2
3	Lê Minh Cường	2

Giải thích logic: Tiêu chí phụ g.guide_id ASC phân xử trường hợp đồng hạng 2 chuyến giữa các nhân sự.

6 . Tính tổng số tiền từng khách đại diện đã thanh toán trong năm 2025 theo payment_date; hiển thị cả người chưa trả tiền với tổng bằng 0

Mã SQL:

```
SELECT      c.customer_id,      c.full_name      AS      customer_name,
COALESCE(SUM(p.amount), 0) AS total_paid
FROM customers c
    LEFT JOIN trips t ON t.customer_id = c.customer_id
    LEFT JOIN payments p ON p.trip_id = t.trip_id
        AND p.payment_date >= '2025-01-01'
        AND p.payment_date <  '2026-01-01'
GROUP BY c.customer_id, c.full_name
ORDER BY c.customer_id;
```

Bảng kết quả:

customer_id	customer_name	total_paid (VND)
1	Đặng Quốc Huy	12.100.000
2	Bùi Thanh Lan	2.500.000
3	Ngô Minh Khoa	5.000.000
4	Đỗ Hồng Nhung	6.000.000
5	Phan Gia Bảo	4.400.000
6, 7, 8, 9, 10	Lý Mỹ Linh, Trịnh Tuấn Kiệt, Mai Phương Thảo, Hồ Đức Thịnh, Dương Ánh Tuyết	0

Giải thích logic: Hàm COALESCE đưa các giá trị NULL về 0 cho những khách hàng chưa có giao dịch trong năm 2025. Khách 6 (Lý Mỹ Linh) thanh toán năm 2024 nên tổng năm 2025 chuẩn xác bằng 0.

7 . Liệt kê các khách đại diện chưa có bất kỳ bản ghi thanh toán nào trong toàn bộ dữ liệu, kể cả người chưa có giao dịch

Mã SQL:

```
SELECT c.customer_id, c.full_name AS customer_name
FROM customers c
    LEFT JOIN trips t ON t.customer_id = c.customer_id
    LEFT JOIN payments p ON t.trip_id = p.trip_id
GROUP BY c.customer_id, c.full_name
HAVING COUNT(p.payment_id) = 0
ORDER BY c.customer_id;
```

Bảng kết quả:

customer_id	customer_name
7	Trịnh Tuấn Kiệt
8	Mai Phương Thảo
9	Hồ Đức Thịnh
10	Dương Ánh Tuyết

Giải thích logic: Điều kiện HAVING COUNT(p.payment_id) = 0 quét toàn bộ lịch sử thanh toán của khách hàng.

8 . Liệt kê toàn bộ khách đại diện theo tổng phí hướng dẫn đã xác nhận giảm dần, không phụ thuộc đã thanh toán hay chưa; nếu bằng nhau, sắp theo mã tăng dần

Mã SQL:

```
SELECT c.customer_id, c.full_name AS customer_name, COALESCE(SUM(r.fee),
0) AS total_fee
FROM customers c
    LEFT JOIN trips t ON t.customer_id = c.customer_id
    LEFT JOIN trip_reports r ON r.trip_id = t.trip_id
GROUP BY c.customer_id, customer_name
ORDER BY total_fee DESC, c.customer_id ASC;
```

Bảng kết quả:

customer_id	customer_name	total_fee (VND)
1	Đặng Quốc Huy	12.900.000
4	Đỗ Hồng Nhung	6.000.000
3	Ngô Minh Khoa	5.000.000
5	Phan Gia Bảo	4.400.000
6	Lý Mỹ Linh	3.200.000
10	Dương Ánh Tuyết	3.000.000
2	Bùi Thanh Lan	2.500.000
7, 8, 9	Trịnh Tuấn Kiệt, Mai Phương Thảo, Hồ Đức Thịnh	0

9: Tính tỷ lệ hủy của từng hướng dẫn viên trong năm 2025: số chuyến tham quan Cancelled chia tổng số chuyến tham quan của năm, nhân 100 (Làm tròn 2 chữ số; trả NULL nếu tổng bằng 0)

Mã SQL:

```
SELECT g.guide_id, g.full_name AS guide_name,
       ROUND(SUM(CASE WHEN t.status = 'Cancelled' THEN 1 ELSE 0 END) /
NULLIF(COUNT(t.trip_id), 0) * 100, 2) AS cancel_rate
FROM guides g
    LEFT JOIN trips t ON t.guide_id = g.guide_id
        AND t.departure_date >= '2025-01-01'
        AND t.departure_date < '2026-01-01'
GROUP BY g.guide_id, g.full_name
ORDER BY g.guide_id;
```

Bảng kết quả:

guide_id	guide_name	cancel_rate (%)
1	Nguyễn Văn An	0
2	Trần Thị Bình	3.333
3	Lê Minh Cường	0
4	Phạm Thu Dung	0
5	Hoàng Văn Em	0
6	Vũ Thị Hoa	*NULL*

Giải thích logic: Hàm NULLIF(COUNT(t.trip_id), 0) khử lỗi chia cho 0 (division by zero) đối với hướng dẫn viên Vũ Thị Hoa (không có chuyến nào năm 2025, trả về NULL).

10: Liệt kê các khách đại diện có hơn 3 chuyến tham quan Completed trong năm 2025, kèm số lần; sắp số lần giảm dần

Mã SQL:

```
SELECT c.customer_id, c.full_name AS customer_name, COUNT(t.trip_id) AS
completed_trips
FROM customers c
    JOIN trips t ON t.customer_id = c.customer_id
WHERE t.status = 'Completed'
    AND t.departure_date >= '2025-01-01'
    AND t.departure_date < '2026-01-01'
GROUP BY c.customer_id, c.full_name
HAVING COUNT(t.trip_id) > 3
ORDER BY completed_trips DESC, c.customer_id ASC;
```

Bảng kết quả:

customer_id	customer_name	completed_trips
1	Đặng Quốc Huy	4

Giải thích logic: Mệnh đề HAVING COUNT(t.trip_id) > 3 lọc đúng nhóm đối tượng có số chuyến hoàn thành trên 3 (kết quả trích xuất duy nhất khách hàng Đặng Quốc Huy).

PHẦN 5: KẾT LUẬN VÀ ĐÁNH GIÁ MỞ RỘNG

1 . Tính toàn vẹn nghiệp vụ: Lược đồ thiết kế đạt chuẩn 3NF, tận dụng triệt để khóa ngoại phức hợp (trip_id, status) và các mệnh đề CHECK để bảo vệ toàn vẹn dữ liệu từ tầng lõi cơ sở dữ liệu.

2 . Khả năng mở rộng: Trong môi trường thực tế với hàng triệu bản ghi, hệ thống cần đánh chỉ mục B-Tree trên các trường departure_date, payment_date và áp dụng kỹ thuật phân vùng bảng (Partitioning) theo năm để tối ưu tốc độ xử lý các truy vấn tổng hợp định kỳ.